

Toolkit 2.1 Guide

Contents

1	Overview	3
1.1	Motivation	3
2	Getting Started	3
2.1	Required Packages	3
2.2	Loading the Toolkit	3
3	Measures	4
3.1	Weight	4
3.2	Grain Size	4
3.3	Probabilities and Frequencies	5
3.4	PP Measures	5
3.5	Measure Naming	6
4	Datasets	7
4.1	Corpus and Mapping Datasets	7
4.2	Mapping Data	7
4.3	Word Data	7
5	Functions	7
5.1	Preprocessing Utilities	8
5.2	Mapping Functions	8
5.3	Functions for Data Processing	10
6	Advanced Usage	15
6.1	Full Unit Table	15
6.1.1	Functions	15
6.2	Generating All Toolkit Measures	15
6.2.1	Code Walkthrough	16
6.2.2	Output	17
6.2.3	Usage	17
6.3	Map-Words Pipeline	17
6.3.1	Preliminaries	18
6.3.2	Functions	18
6.3.3	Usage	20
6.4	Parsing Probabilities Pipeline	20
6.4.1	Running parse_corpus	20
6.4.2	Inspecting the Outputs	21
6.4.3	Querying PP for Specific Words	21
6.5	Generating Pseudoword Spellings and Pronunciations	21
6.5.1	Generating Spellings with pw_spell	21
6.5.2	Generating Pronunciations with pw_read	22
6.5.3	Adjusting Thresholds	22
6.5.4	Interpreting pw_read Output	23
6.6	Full Toolkit Pipeline	23

6.7	Generating Pseudoword Measures	24
6.8	Obtaining Number of Units	24
6.9	Finding Orthographic and Phonological Neighbors	24
7	Troubleshooting	26
7.1	Function Hangs without Error	26
8	Version 2.1 Updates	27
8.1	New Wordlist	27
8.2	summarize_words — ONC Mode	27
8.3	Simplified min_map Parameter	27
8.4	visualize_parse_tree (New Function)	27
9	Appendix	28
9.1	Phoneme Reference	28

1 Overview

This guide will provide a general overview of the Toolkit, its various functions, and common usage. For more details and relevant files/scripts, see the GitHub repository. If you don't have access to the repository and should, please email me at cjsolo5@umd.edu. Please also email me if you find any typos, errors, or other areas where the guide could be improved.

1.1 Motivation

The English Sublexical Toolkit¹ is a package of R functions and datasets for processing and understanding English words at various levels of analysis. It currently supports 22,257 unique English words, and contains functionality for computing all relevant measures for any provided set of valid words or pseudowords (words containing valid sequences of respective phonemes and graphemes)². The Toolkit also has its own conventions for representing phonemes; see Appendix 3.1 for reference.

2 Getting Started

2.1 Required Packages

The following packages must be installed:

```
stringr  
stringi  
dplyr  
readxl  
progress  
data.tree
```

2.2 Loading the Toolkit

It will be easier to follow along with the following sections if you have an active R environment with the Toolkit loaded. Navigate to a directory of your choice and either open the R REPL or a new R file, and set your working directory to the folder containing the `Toolkit_v2.1.RData` file:

```
setwd("path/to/directory")
```

Then simply call

```
load("Toolkit_v2.1.RData")
```

It will take a few moments for the file to load in to your environment. Once this completes and a new terminal prompt appears, you are ready to use the Toolkit.

With the RData file loaded, you'll be able to see all datasets and functions using the `view` function. *PROCESSING_SCRIPTS.R* is the source file for the Toolkit.

¹The English Sublexical Toolkit: Methods for indexing sound-spelling consistency. <https://doi.org/10.3758/s13428-024-02395-3>

²A "valid" phoneme-grapheme mapping is one that is supported in the Toolkit at the desired level. Novel mappings are added as they are encountered in the corpus, which continues to grow with Toolkit updates.

3 Measures

For the various measures computed by the Toolkit, there are two discriminating categories: *weight* and *grain size*. For each combination of these, a word has various *frequency* and *consistency* measures. Words also have *parsing probability* (PP) measures.

3.1 Weight

Measures in the Toolkit are either *type-weighted* (where frequency of the word in the corpus is not taken into account) or *token-weighted* (where frequency of the word in the corpus is provided by the user). In the Toolkit's data, preprocessed datasets involving the suffix "_freq" indicate token-weighted measures, and those without are type-weighted. Token-weighted Toolkit measures are weighted by subtitle frequency.

Measures can be further weighted based on the position of the mapping in the word; by default, this is how measures are calculated (see the relevant functions in the Functions section). Tables named with the suffix "_noposition" indicate that their measures disregard the position of the mapping in the word when accounting for frequency.

To illustrate this:

- `scored_words_PG` has data for all words mapped at the phoneme-grapheme level, type-weighted and taking into account mapping position.
- `scored_words_PG_freq` has data for phoneme-grapheme-mapped words, token-weighted and taking into account mapping position.
- `scored_words_PG_noposition` has data for phoneme-grapheme-mapped words, type-weighted and disregarding mapping position.
- `scored_words_PG_freq_noposition` has data for phoneme-grapheme-mapped words, token-weighted and disregarding mapping position.

The case is analogous for all other datasets. By convention, in token-weighted position-disregarding datasets the "_freq" will come before the "_noposition" in dataset naming.

3.2 Grain Size

The Toolkit currently supports four different *grain sizes*, units of granularity for understanding phoneme-grapheme mappings in words. These are indicated in table names as the following:

- **PG:** phoneme-grapheme level.
The phoneme-grapheme grain size is the smallest possible level of analysis for a word. A phoneme is a single phonological unit, or sound, and a grapheme is the collection of one or more letters corresponding to the phoneme in a word.
- **OR:** onset-rime level.
The *onset* is the initial consonant or consonant cluster of a syllable (and its corresponding phonological representation), while the *rime* is the subsequent vowel and remainder of the syllable.

- **OC**: oncleus-coda level.

The *oncleus* is the combination of the *onset* (initial consonant cluster) and *nucleus* (internal sound between the initial and final consonant cluster) for a syllable, while the *coda* is any remaining sounds within the same syllable.

- **ONC**: onset-nucleus-coda level.

Consideration of the onset, nucleus, and coda separately.

3.3 Probabilities and Frequencies

The Toolkit provides five different measures (and each can be weighted by any of the options previously discussed, for any of the grain sizes). These are:

- **PG**: spelling consistency; $p(G|P)$, the probability of a spelling G given phonemes P
- **GP**: reading consistency; $p(P|G)$, the probability of phonemes P given a spelling G
- **PG_freq**: the $\log_{10}$ frequency of the correspondence between pronunciation P and spelling G , identical in both the spelling and reading directions
- **P_freq**: the $\log_{10}$ frequency of the phoneme P
- **G_freq**: the $\log_{10}$ frequency of the grapheme G

3.4 PP Measures

PP, or *parsing probability* measures, describe the likelihood that a sequence of letters will be grouped into a specific grapheme. For each letter in a word, PP quantifies the possible ways that letter could be parsed as part of a grapheme, and how probable each parsing is based on the corpus.

The Toolkit computes PP using the `parse_corpus` function, which builds a tree of all possible grapheme parsings for every word in the corpus. For words ending in ED, ER, or ES, a suffix class flag is applied to condition the PP statistics on whether a word belongs to this morphological class (the “EDERES” class) or not (the “OTHER” class).

There are 4 PP measures available. These are type- and token-weighted mean and min values:

pp_mean, pp_min, pp_freq_mean, pp_freq_min

- **pp_mean**: The mean parsing probability across all letters in a word (type-weighted).
- **pp_min**: The minimum parsing probability across all letters in a word (type-weighted).
- **pp_freq_mean**: The mean parsing probability across all letters in a word (token-weighted by $\log_{10}(\text{frequency} + 1)$).
- **pp_freq_min**: The minimum parsing probability across all letters in a word (token-weighted by $\log_{10}(\text{frequency} + 1)$).

The `parse_corpus` function returns four outputs:

- `[[1]] pp_table`: The parsing probabilities for each letter-grapheme combination.
- `[[2]] pp_reduced`: Reduced choice sets indicating which grapheme options actually occur in the corpus. This is needed by `pw_read`.
- `[[3]] corpus_parsed`: The full breakdown of words into letters and how they are assigned to graphemes, with PP values.
- `[[4]] corpus_pp`: Per-word summary with `pp_mean`, `pp_min`, `pp_freq_mean`, and `pp_freq_min`. This is the most useful output to inspect.

See Section 5 for full function documentation and Section 6 for usage examples. Pre-computed PP measures are also available in *data/precalculated/pp-measures.csv*.

3.5 Measure Naming

Measures are named according to the following scheme:

`<grain_size>.<weight>.<measure>.<statistic>`

Where

- *grain_size* corresponds to one of the grain sizes mentioned above in Section 3.2,
- *weight* corresponds to a weighting style as discussed in Section 3.1 (note for the default weighting with type-weighting and position considered, this field will be `"_default"`),
- *measure* corresponds to one of the probabilities or frequencies outlined in Section 3.3, and
- *statistic* corresponds to one of the following: mean, median, max, min, sd (standard deviation).

To take a few examples:

- `PG_default_PG.mean`
The average conditional probability of phoneme-to-grapheme mappings for position-constrained phoneme-grapheme units, not weighted by corpus frequency.
- `ONC_noposition_GP.max`
The highest conditional probability of grapheme-to-phoneme mappings for position-unconstrained onset-nucleus-coda units, not weighted by corpus frequency.
- `OC_freq_PG_freq.min`
The lowest frequency of phoneme-grapheme units for position-constrained oncleus-coda units, weighted by corpus frequency.
- `OR_freq_noposition_P_freq.median`
The median frequency of phoneme units for position-unconstrained onset-rime units, weighted by corpus frequency.

4 Datasets

With the Toolkit loaded in your R environment, call `ls()` in your terminal to see the full list of datasets available from the Toolkit.

4.1 Corpus and Mapping Datasets

`wordlist_v2_1_merged`: Contains all English words in the Toolkit.

`<type>_mappings`: Lists of acceptable phoneme-grapheme mappings for various positions. Replace `<type>` with your desired position to obtain the relevant list. The options are *syllable_initial*, *syllable_medial*, *syllable_final*, *word_initial*, and *word_final*. For instance, calling `head(syllable_initial_mappings)` will show us the first six valid phoneme-grapheme mappings in the syllable-initial position (recall that all phonemes are represented using the Toolkit's conventions):

```
> head(syllable_initial_mappings)
  phoneme grapheme
1      @         a
2      @        a_e
3      @        i_e
4      5         i
5      5        i_e
6      0        ou
```

4.2 Mapping Data

The `all_tables` family of precomputed datasets contains all probabilities and frequencies for each word individually at the desired grain size and weight (see the Measures section), while the `all_words` family of datasets contains all phoneme-grapheme mappings for all words at the desired grain size.

4.3 Word Data

The Toolkit has pre-computed datasets for all combinations of weight and grain size, containing all measures for these parameters for each of the 22,257 words. These are the `scored_words` family of datasets. See the Measures section for relevant information on what various options for these datasets means, and what measures are provided within.

5 Functions

With the Toolkit loaded in your R environment, call `ls()` in your terminal to see the full list of functions available. These will be listed with the datasets discussed above.

The Toolkit has a set of functions allowing one to both 1) reproduce the Toolkit's results and 2) calculate measures for any chosen set of valid phoneme-grapheme mappings at a desired grain size.

5.1 Preprocessing Utilities

`replace_accented_vowels(text)`:

`text`: A string or character vector of spellings.

Strips accented vowels from spellings before processing. Replaces characters such as á, é, ó, ú, à, è, ì, ò, ù, ä, ë, ï, ö with their unaccented equivalents. This should be applied to spellings before passing them to any mapping function:

```
wordlist_v2_1_merged$spelling <-
  replace_accented_vowels(wordlist_v2_1_merged$spelling)
```

5.2 Mapping Functions

`map_PG(spelling, pronunciation, map_progress=FALSE)`: Given a valid *spelling* and *pronunciation* (in the Toolkit format), maps the two at the phoneme-grapheme level, showing the live progress as desired. Accepts arguments as lists (enabling words to be batch-mapped). One could, for instance, reproduce the `all_words_PG` dataset by calling the following:

```
all_words_PG <- map_PG(spelling=wordlist_v2_1_merged$spelling,
  pronunciation = wordlist_v2_1_merged$pronunciation)
```

Or simply map a single word to a pronunciation:

```
map_PG("toolkit", "tUlkit")
```

This would yield a result of the form:

```
> map_PG("toolkit", "tUlkit")
[[1]]
[[1]][[1]]
  X6 X5 X4 X3 X2 X1
1  t  U  1  k  1  t
2  t oo  1  k  i  t
3  1  3  4  2  3  5

[[2]]
  [,1]
[1,] TRUE
```

Notice that there will always be two lists of results returned from this (and all other `map`) function; one with the mapping as a dataframe, and the other indicating the success. For all other `map` functions, usage is the same.

`map_ONC(spelling, pronunciation, map_progress=FALSE)`: Maps a valid *spelling* and *pronunciation* at the onset-nucleus-coda level.

`map_OC(spelling, pronunciation, map_progress=FALSE)`: Maps a valid *spelling* and *pronunciation* at the oncleus-coda level.

`map_OR(spelling, pronunciation, map_progress=FALSE)`: Maps a valid *spelling* and *pronun-*

ciation at the onset-rime level.

```
make_tables(mapped_words, weight=FALSE, positional=TRUE):
```

mapped_words: A table produced by one of the `map` functions.

weight: If `FALSE`, results will be type-weighted; otherwise, if a list of frequencies from a corpus is provided, results will be token-weighted based on frequency. Frequencies for weight from the corpus used in the Toolkit are obtained from `wordlist_v2_1_merged$freq`. This list must be of the same length as the number of entries passed in to the respective `map` function and should correspond to the relevant words, in order.

positional: If `FALSE`, mapping position will be disregarded when measures are computed.

The four types of `all_tables` datasets are generated using this function. Unless one is using their own corpus of words, it's unnecessary to use this function; however, the Toolkit's results can be reproduced by the following:

```
all_tables_PG <- make_tables(all_words_PG)
all_tables_PG_noposition <- make_tables(all_words_PG, positional=FALSE)
all_tables_PG_freq <-
  make_tables(all_words_PG, weight=wordlist_v2_1_merged$freq)
all_tables_PG_freq_noposition <-
  make_tables(all_words_PG,
              weight=wordlist_v2_1_merged$freq,
              positional=FALSE)
```

Usage is analogous for all other grain sizes. Results can be indexed as follows (using `all_tables_PG` as an example):

```
> head(all_tables_PG[[1]])
```

	phoneme	grapheme	wi	si	sm	sf	wf
1	@	a	0.9823	0.9556	0.9544	0.9964	0.3333
2	@	a_e	0.0088	0.0333	0.0414	0	0
3	@	aa	0	0	0	0	0.3333
4	@	aha	0	0	6e-04	0	0
5	@	ai	0	0	6e-04	9e-04	0
6	@	au	0.0088	0	0.003	0.0018	0

As in the PG case, there are 527 valid mappings whose frequencies are produced for every word. The first six are shown here.

```
map_value(spelling, pronunciation, level, tables, progress=FALSE):
```

spelling: A string or list of strings of words.

pronunciation: A string (if `spelling` is a single string) or a list of the same length as `spelling` of corresponding strings representing the corresponding pronunciations.

level: The desired grain size at which to map; valid string inputs are `PG`, `OR`, `OC`, and `ONC`.

tables: A table of values produced by `make_tables` corresponding to the grain size specified in `level`, depending on the desired weight.

progress: Whether or not to display a progress bar during computation.

Outputs a list of matrices, where each matrix contains the mapping, probabilities, and frequencies for its respective word at the level mapped. In the preprocessed Toolkit, outputs are the `scored_words` family of datasets (see the Datasets section). To reproduce these (taking PG as an example), perform:

```
scored_words_PG <-
  map_value(spelling=wordlist_v2_1_merged$spelling,
            pronunciation=wordlist_v2_1_merged$pronunciation,
            level="PG", tables=all_tables_PG)
```

Other datasets, such as `scored_words_PG_noposition`, are generated by passing a relevantly-weighted `all_tables` dataset (for instance, `all_tables_PG_noposition`). We can also map an individual word at a desired level and weight; try:

```
map_value("toolkit", "tUlk1t", "PG", all_tables_PG)
```

Which yields:

```
> map_value("toolkit", "tUlk1t", "PG", all_tables_PG)
===== [[1]]
      toolkit    NA    NA    NA    NA    NA
phoneme      t      U      l      k      1      t
grapheme      t      oo     l      k      i      t
position      wi     sm     sf     si     sm     wf
PG           0.9942 0.5317 0.8682 0.2059 0.7859 0.8302
GP            1 0.3323      1      1 0.8159      1
PG_freq       3.0111 2.0414 2.6522 2.5527 3.5053 3.2947
P_freq        3.0137 2.3139 2.7135 3.238 3.6099 3.3755
G_freq        3.0111 2.5172 2.6522 2.5527 3.5936 3.2947
spelling      toolkit
pronunciation  tUlk1t
PG_accuracy    TRUE
```

See the Measures section for a description of each of the provided values.

5.3 Functions for Data Processing

```
summarize_words(mapped_words, parameter, mode="default"):
```

mapped_words: A list of words from `wordlist_v2_1_merged` or an output of `map_value` corresponding to the desired word(s) of interest

parameter: The desired measure to be obtained: *PG*, *GP*, *PG_freq*, *P_freq*, or *G_freq*

mode: If "default", summarizes across all units. If "ONC", provides a breakdown by syllabic position (onset, nucleus, coda). See the Version 2.1 Updates section for details.

Outputs a matrix of statistics for the desired measure across the inputs. We can see this for a single word, for instance, the various statistics (mean, median, max, min, and sd) for the word "toolkit"’s spelling consistency at the PG level:

```
summarize_words(
  mapped_words =
    scored_words_PG[which(wordlist_v2_1_merged$spelling=="toolkit")],
  parameter = "PG")
```

```
> summarize_words(mapped_words = scored_words_PG[which(wordlist_v2_0$spelling=$
  spelling pronunciation      mean median  max   min    sd
1 toolkit      tUlkit 0.7026833 0.80805 0.9942 0.2059 0.2869119
```

The `all_words_<direction>_probability` family of datasets is also generated using this function:

```
all_words_PG_probability <-
  summarize_words(
    mapped_words = scored_words_PG,
    parameter = "PG")
```

```
head(all_words_PG_probability)
```

```
> head(all_words_PG_probability)
  spelling pronunciation      mean median  max   min    sd
1      a             8 0.6667000 0.66670 0.6667 0.6667    NA
2      a             e 0.8409000 0.84090 0.8409 0.8409    NA
3     aah            a 0.0041000 0.00410 0.0041 0.0041    NA
4 aardvark      ardvar 0.6839286 0.96780 0.9959 0.0041 0.4037696
5   aaron       Eren 0.5082750 0.52230 0.9861 0.0024 0.4597517
6  abacus      @bekes 0.5613333 0.57895 0.9823 0.0483 0.3702818
```

```
parse_corpus(corpus, mapped_corpus):
```

`corpus`: A wordlist dataframe with `spelling`, `pronunciation`, and `freq` columns (e.g., `wordlist_v2_1_merged`).

`mapped_corpus`: The output of `map_PG()` on the same corpus (e.g., `all_words_PG`).

Builds a tree of all possible grapheme parsings for every word in the corpus, computes parsing probabilities (PP), and returns a list of four elements:

- `[[1]] pp_table`: Parsing probabilities for each letter-grapheme combination.
- `[[2]] pp_reduced`: Reduced choice sets (which grapheme options actually occur in the corpus). Used in `pw_read()`.
- `[[3]] corpus_parsed`: Full word-letter-grapheme breakdown with PP values merged in.
- `[[4]] corpus_pp`: Per-word summary with `pp_mean`, `pp_min`, `pp_freq_mean`, and `pp_freq_min`.

Requires: `progress` and `data.tree` packages.

Usage:

```
parsed_corpus <- parse_corpus(wordlist_v2_1_merged, all_words_PG)
```

Inspect the outputs:

```
head(parsed_corpus[[1]]) # pp_table
head(parsed_corpus[[2]]) # pp_reduced
head(parsed_corpus[[3]]) # corpus_parsed
head(parsed_corpus[[4]]) # corpus_pp
```

Query PP for a specific word:

```
parsed_corpus[[4]][parsed_corpus[[4]]$spelling == "yacht", ]
```

Query per-letter PP breakdown:

```
parsed_corpus[[3]][parsed_corpus[[3]]$spelling == "yacht", ]
```

```
word_pattern(mapped_words, phoneme, grapheme, position):
```

mapped_words: An output dataset from `map_value`. The `scored_words` family of datasets works here.

phoneme: Desired phoneme in the Toolkit's format. *any* is a valid string input here if one desires only a grapheme corresponding to any pronunciation at the desired position.

grapheme: Desired corresponding grapheme. *any* is a valid string input here if one desires only a phoneme corresponding to any grapheme at the desired position.

position: Position of the phoneme-grapheme mapping in the word; valid string inputs to this are *si* (syllable-initial), *sm* (syllable-medial), *sf* (syllable-final), *wi* (word-initial), and *wf* (word-final). *any* is a valid input here if one desires the specified phoneme-grapheme mapping but not in a specific position.

Outputs a list of all words present in the corpus containing the specified phoneme-grapheme mapping at the desired position. For instance, if we'd like to obtain a list of all words beginning with the grapheme "ho" (meaning "ho" appears in the *wi* word-initial position), irrespective of pronunciation, we call:

```
word_pattern(scored_words_PG,"any","ho","wi")
```

Which outputs:

```
> word_pattern(scored_words_PG,"any","ho","wi")
[1,]
[1,] "homage"
[2,] "honest"
[3,] "honestly"
[4,] "honesty"
[5,] "honor"
[6,] "honorable"
[7,] "honorary"
[8,] "honored"
[9,] "honoring"
[10,] "honors"
[11,] "honour"
[12,] "honourable"
[13,] "honoured"
[14,] "honours"
[15,] "hors"
[16,] "hour"
[17,] "hourglass"
[18,] "hours"
```

```
pw_spell(target, level, score, PG_table, OC_table, OR_table,
          min_map, max_options, param, mean_score, parsed_corpus,
```

summary_mode):

Generates possible spellings for a given pronunciation string. See Section 6.5 for usage examples.

target: A pronunciation string (or vector of strings) in the Toolkit's phonemic notation (e.g., "sib", "f15k").

level: The grain size at which to generate spellings. Valid values: "PG", "OC", "OR", or "all". When "all", the function returns a list with results at every level. Default: "PG".

score: If TRUE (default), the generated spellings are mapped and scored using `map_value` and `summarize_words`, returning a data frame of spellings with their consistency scores. If FALSE, returns only the raw spelling strings without scoring.

PG_table: A table produced by `make_tables` at the PG level (e.g., `all_tables_PG`). Required when `level = "PG"` or `level = "all"`. Default: NULL.

OC_table: A table produced by `make_tables` at the OC level (e.g., `all_tables_OC`). Required when `level = "OC"` or `level = "all"`. Default: NULL.

OR_table: A table produced by `make_tables` at the OR level (e.g., `all_tables_OR`). Required when `level = "OR"` or `level = "all"`. Default: NULL.

min_map: Minimum mapping probability threshold for including a grapheme option. A value of 1 returns only the most probable spelling(s); lower values (e.g., 0.01) include less probable options above the threshold; 0 includes all valid options. Default: 1.

max_options: Maximum number of spelling candidates to generate per target before adaptive thresholding is applied. If the number of candidates exceeds this limit, `min_map` is progressively raised to reduce the set. Default: 1000.

param: The measure used for scoring when `score = TRUE`. Valid values: "PG", "GP", "PG_freq", "P_freq", "G_freq", or "pp". When "pp", parsing probabilities are used for scoring (requires `parsed_corpus` and `PG_table`). Default: "pg".

mean_score: Minimum mean score threshold for filtering scored results. A value of 1 returns only the highest-scoring spelling; values between 0 and 1 return all spellings whose mean score meets the threshold; 0 (default) returns all scored spellings without filtering. Only available when `summary_mode = "default"`.

parsed_corpus: The output of `parse_corpus()`. Required only when `param = "pp"` for scoring spellings by parsing probability. Default: NULL.

summary_mode: Controls the format of the scoring output. "default" summarizes across all units (mean, median, max, min, sd). "ONC" provides a breakdown by syllabic position (onset, nucleus, coda). Default: "default".

```
pw_read(target, parsed_corpus, PG_table, OC_table, OR_table,
        level, score, min_pp, min_map, mean_score, max_options,
        param, summary_mode):
```

Generates possible pronunciations for a spelling string based on the corpus statistics computed by `parse_corpus`. See Section 6.5 for usage examples.

target: A spelling string (or vector of strings) containing only alphabetic characters (e.g., "flope", "sebe").

parsed_corpus: The output of `parse_corpus()`. Required. This provides the grapheme parsing probabilities and reduced choice sets used to determine possible pronunciations.

PG_table: A table produced by `make_tables` at the PG level (e.g., `all_tables_PG`). Required.

OC_table: A table produced by `make_tables` at the OC level (e.g., `all_tables_OC`). Required when `level = "all"`. Default: `NULL`.

OR_table: A table produced by `make_tables` at the OR level (e.g., `all_tables_OR`). Required when `level = "all"`. Default: `NULL`.

level: The grain size at which to generate and score pronunciations. Valid values: `"PG"` or `"all"`. When `"all"`, returns results scored at PG, OC, and OR levels. Default: `"PG"`.

score: If `TRUE` (default), the generated pronunciations are mapped and scored using `map_value` and `summarize_words`, returning consistency scores at the requested level(s). If `FALSE`, returns only the pronunciation candidates and their parsing probability means.

min_pp: Minimum parsing probability threshold for grapheme parses. A value of 1 (default) uses only the most probable grapheme parse for each letter position; lower values include alternative parses above the threshold, yielding more pronunciation candidates.

min_map: Minimum mapping probability threshold for including a phoneme option at each grapheme position. Lower values include less probable phoneme mappings. Default: 0.01.

mean_score: Minimum mean score threshold for filtering the final scored pronunciations. A value of 1 returns only the highest-scoring pronunciation; values between 0 and 1 return all pronunciations meeting the threshold; 0 (default) returns all. Only available when `summary_mode = "default"`.

max_options: Maximum number of pronunciation candidates to retain. When the candidate set exceeds this limit, only the top candidates (ranked by the scoring metric) are kept. Default: 1000.

param: The measure used for scoring when `score = TRUE`. Valid values: `"PG"`, `"GP"`, `"PG_freq"`, `"P_freq"`, `"G_freq"`, or `"pp"`. Default: `"GP"`.

summary_mode: Controls the format of the scoring output. `"default"` summarizes across all units (mean, median, max, min, sd). `"ONC"` provides a breakdown by syllabic position (onset, nucleus, coda). Default: `"default"`.

**visualize_parse_tree(spelling, pronunciation, show_plot,
include_suffix_variants):**

Visualizes how a spelling can be parsed into graphemes, building a tree of all possible grapheme parsings and optionally highlighting the parse path that corresponds to a given pronunciation. See Section 8.4 for usage examples. *Requires:* `data.tree` and `igraph` packages (the latter only when `show_plot = TRUE`).

spelling: A single spelling string to parse (e.g., `"sebe"`, `"reach"`).

pronunciation: A pronunciation string in the Toolkit's phonemic notation (e.g., `"sib"`). If provided, the function identifies which parse branch(es) match this pronunciation and highlights them in the plot. If `NULL` (default), all branches are shown without highlighting.

show_plot: If `TRUE`, renders the parse tree as a graph using `igraph`, with nodes colored gold and edges colored red for branches matching the given pronunciation. If `FALSE` (default), returns the parse branches as a character vector without plotting.

include_suffix_variants: If `TRUE` (default), the function also considers variant parsings for inflectional suffixes. For spellings ending in `-es` or `-ed`, additional branches are generated by parsing the base form (without the final letter) and appending the suffix as a separate unit. This captures cases where the suffix letter is not part of the preceding grapheme.

6 Advanced Usage

This section will describe more advanced usage of the Toolkit, through scripts and functions not directly part of the Toolkit itself. Follow along with the files in the repository. Some Python will be used and discussed here as well, primarily in the handling of datasets generated from the Toolkit in R.

6.1 Full Unit Table

Here, we discuss how to generate a spanning table of all Toolkit units for a corpus of words, using the functions in *scripts/all-units/unit.table.R*. This script provides tools for extracting phoneme-grapheme pairs at any grain size and optionally converting from the Toolkit's in-house phoneme encoding to IPA.

6.1.1 Functions

```
extract_pg_pairs(scored_words, spellings=NULL, col_prefix="PG",
                 toolkit_to_ipa=FALSE)
```

scored_words: A dataset of the `scored_words` family (output from the Toolkit's `map_value` function).

spellings: An optional list of spellings of the same length as the `scored_words` dataset, used for cross-verification.

col_prefix: The grain size designation for column naming. Acceptable entries: *PG*, *OR*, *OC*, *ONC*, *SY* (syllable).

toolkit_to_ipa: If `TRUE`, phonemes in the output will be converted from Toolkit encoding to IPA.

This function iterates through the scored words dataset, extracts all phoneme-grapheme pairs formatted as **phoneme+grapheme**, and builds a padded dataframe. For the *OR* and *SY* grain sizes, it further classifies units into onsets and rimes. For *ONC*, it classifies units into onset, nucleus, and coda components per syllable. Internal helper functions `or_classifier` and `onc_classifier` handle unit classification using the Toolkit's vowel and consonant inventories.

```
extract_units_at_level(wordlist, level)
```

wordlist: A dataframe with columns `spelling`, `pronunciation`, and `freq`.

level: The desired grain size (*PG*, *OR*, *OC*, or *ONC*).

A wrapper function that maps all words in the wordlist at the desired level using the appropriate Toolkit mapping function, abstracting away the need to call `extract_pg_pairs` directly. The script also includes `inhouse_to_ipa` and `ipa_to_inhouse` conversion utilities for translating between the Toolkit's phoneme encoding and IPA.

6.2 Generating All Toolkit Measures

Here, we discuss how to generate a dataset spanning the entire Toolkit containing every measure for every word, at every grain size and weight combination. The full code for this is provided in

scripts/all-measures/extract_all_measures.R - follow along there. Executing this script as it is will generate the *all_measures.csv* dataset to the same folder (and the precomputed version is present in the *data/precalculated* folder).

6.2.1 Code Walkthrough

First, four lists are defined:

- **grain_sizes**: Keys of this list are the grain sizes currently available in the Toolkit, and values are the corresponding default dataset (type-weighted and positional) of the **scored_words** family, which is indexed when measures are obtained.
- **weight_options**: Keys of this list correspond to the various combinations of weight options currently available in the Toolkit. "default" refers to type-weighted positional measures. Values are the corresponding string suffixes to be appended to dataset names in order to index the proper respective dataset of the **scored_words** family. More on this below in the function breakdown.
- **measures**: The five measures available in the Toolkit.
- **statistics**: The five statistics available for each measure in the Toolkit.

Removing an entry from any of these lists will cause respective measures to be omitted from the dataset that is generated. Adjust these lists as necessary to fit the needs of your project.

Next, we define three functions:

- **extract_summary(grain, weight, measure)**
Extracts the summary (all of the Toolkit's statistics) for the provided **measure** from the dataset named by the concatenation of **grain** and **weight**.

For instance, if **grain** is *scored_words_PG*, **weight** is *_noposition* and **measure** is *P_freq*, this function will return all of the statistics for *P_freq* from *scored_words_PG_noposition*.

- **filter_columns(df, statistics)**
Removes all statistic columns from a dataframe that do not correspond to the statistics desired. Does so by keeping columns corresponding to statistics present in the passed-in **statistics** list.
- **get_measures(grain_sizes, weight_options, measures, statistics)**
Returns a dataframe with all of the desired **statistics** for all of the desired **measures** from all datasets indexed by combinations of **grain_sizes** and **weight_options**.

The final section of the code simply writes the generated dataset to a file. To include PP measures, run **parse_corpus** on the mapped corpus and merge the resulting **corpus_pp** into the output (see the Full Toolkit Pipeline section).

6.2.2 Output

By convention, columns in the output are named

`<grain_size>_<token?>_<position?>_<measure>_<statistic>`

where:

- **grain_size**: PG, OR, OC, or ONC. Delineates the grain size with respect to which the measure was calculated.
- **token?**: If the measure is token-weighted, will be `"_freq"`. If the measure is type-weighted and position is included, will be `"_default"`.³
- **position?**: Blank if the measure is weighted by position. If position is disregarded, is `"_noposition"`.
- **measure**: The measure described. PG, GP, PG_freq, P_freq, or G_freq.
- **statistic**: What value of the measure is described. Will be mean, median, max, min, or sd.

With this generated, you can either continue to work with the dataset in R, or use another language of your choice and load the csv into a dataframe; for instance, Python with `pandas`.

6.2.3 Usage

To use this script for other projects (for instance, your own corpus as computed by the `map_value` function which outputs datasets of the `scored_words` family), simply adjust the values of the `grain_sizes` and `weight_options` lists to match the naming of your datasets, so that they are properly concatenated and indexed by the functions above.

6.3 Map-Words Pipeline

Here, we present a pipeline of functions to process words in batch through the three main layers of the Toolkit pipeline: `map_words`, `make_tables`, and `map_value`. For the relevant code, see *scripts/batch-mapping/batch-mapping-pipeline.R*.

The purpose of these functions is to enable one to generate datasets of the same format, both in content and naming scheme, to datasets of the `all_words`, `all_tables`, and `scored_words` families from the Toolkit, and create them for immediate use in the environment. This is useful because it generally becomes tedious to individually name variables through script when, for instance, one desires to map a list of words at every grain size individually (PG, OR, OC, ONC) and then use the Toolkit's `make_tables` and `map_values` functions on the results. The functions described here will enable this dataset generation in three steps.

³This may be changed soon to remove the `"default"` indicator.

6.3.1 Preliminaries

First, we denote a convention that must be followed for inputs to these functions. Values of `grain_sizes`:

```
grain_sizes <- list(
  "PG" = "_PG",
  "ONC" = "_ONC",
  "OC" = "_OC",
  "OR" = "OR",
)
```

and `weight_options`:

```
weight_options <- list(
  "default" = "",
  "noposition" = "_noposition",
  "freq" = "_freq",
  "freq_noposition" = "_freq_noposition",
)
```

must remain exactly as shown in order to properly index the appropriate functions from the Toolkit. Entries can be removed as necessary to avoid generation of mappings/calculation of values at undesired levels. These values will also be the suffixes of the relevant datasets (discussed below) for clarity in accordance with the Toolkit's naming format. It is also suggested that keys remain the same, as these same structures can be used later for measure extraction (see *scripts/all_measures/extract_all_measures.R*) and will result in the names of measures in the generated dataset being of the same format as those extracted from the Toolkit's database.

6.3.2 Functions

`batch_map_words(spellings, pronunciations, grain_sizes, output_name)`

spellings: List of spellings

pronunciations: List of pronunciations in Toolkit format of the same length as **spellings**

grain_sizes: List of the form of the `grain_sizes` list

output_name: The root of the variable name to be used for outputs from this function. For instance, if a set of words are to be mapped at the PG and OR levels, and **output_name** is "mwords", then this function will produce the datasets "mwords_PG" and "mwords_OR", saving them to the global environment.

Returns: A list of names of the datasets written to the environment

Calls the Toolkit's `map`-family of functions (`map_PG`, `map_OR`, etc.) on the list of spellings and pronunciations for all levels designated by the passed-in list. Saves outputs to the global environment with **output_name** as the root of the variable name with the appropriate suffix to denote the mapping level. The returned list of dataset names can be used later to batch-remove all generated datasets from the environment using the `rm` command.

```
batch_make_tables(mapped_words_name, grain_sizes, weight_options,
weight_data, output_name)
```

mapped_words_name: The root name of the datasets as produced by the **map** family of Toolkit functions for indexing when creating tables of the relevant grain size. If such datasets were generate by **batch_map_words**, this should have the same value as **output_name** in the call to that function.

grain_sizes: List of grain sizes at which to make tables. Note that datasets from the **map** family of functions must exist in the environment for each desired grain size.

weight_options: List of weight options for tables to be calculated. Supports default, frequency-weighted, nonpositional, and frequency-weighted nonpositional measures.

weight_data: List of frequency data of the same length as and in the same order of the words in the datasets used. If the function is being called on datasets produced by the **batch_map_words** function, this list should be of the same length as and have values corresponding to the entries in the same order of the **spellings** input to that function.

output_name: The root name of the datasets to be written to the global environment

Returns: A list of names of the datasets written to the environment

Calls the Toolkit's **make_tables** function on the appropriately named datasets constructed by the **mapped_words_name** and suffixes as given in **grain_sizes** and **weight_options**, assumed to exist in the environment. Writes outputs of the function to the global environment constructed appropriately from the **output_name** and suffixes as provided in **grain_sizes** and **weight_options**. Uses the values passed to **weight_data** if frequency-weighted tables are desired.

```
batch_map_values(spellings, pronunciations, table_name, grain_sizes,
weight_options, output_name)
```

spellings: A list of spellings for which to obtain measures

pronunciations: A list of the same length as **spellings** of corresponding pronunciations in Toolkit format

table_name: The root name of the datasets as produced by the Toolkit's **make_tables** function to be indexed. If such tables were produced by the **batch_make_tables** function, this value should be the same as the value of **output_name** in the call to that function.

grain_sizes: List of grain sizes at which to obtain measures

weight_options: List of weight options at which to obtain measures

output_name: The root name of the datasets to be written to the global environment

Returns: A list of names of the datasets written to the environment

Calls the Toolkit's **map_value** function on the appropriately named datasets constructed by the **table_name** and suffixes as given in **grain_sizes** and **weight_options**, assumed to exist in the environment. Writes outputs of the function to the global environment constructed appropriately from the **output_name** and suffixes as provided in **grain_sizes** and **weight_options**.

6.3.3 Usage

For a list of spellings **s**, corresponding pronunciations **p**, and corresponding frequency weights **w** with **grain_sizes** and **weight_options** unedited from above, produces the following datasets from the following calls (executed in sequence):

- `l1 <- batch_map_words(s, p, grain_sizes, "mwords")`

Yields to the global environment:

`mwords_PG`, `mwords_OR`, `mwords_OC`, `mwords_ONC`

Contents are of analogous format to the `all_words` family of datasets that exists in the Toolkit.

- `l2 <- batch_make_tables("mwords", grain_sizes, weight_options, w, "mtables")`

Yields to the global environment:

`mtables_PG`, `mtables_PG_noposition`, `mtables_PG_freq`, `mtables_PG_freq_noposition`, and all analogous tables for `OC`, `OR`, and `ONC`

Contents are of analogous format to the `all_tables` family of datasets that exists in the Toolkit.

- `l3 <- batch_map_values(s, p, "mtables", grain_sizes, weight_options, "mvalues")`

Yields to the global environment:

`mvalues_PG`, `mvalues_PG_noposition`, `mvalues_PG_freq`, `mvalues_PG_freq_noposition`, and all analogous tables for `OC`, `OR`, and `ONC`

Contents are of analogous format to the `scored_words` family of datasets that exists in the Toolkit. By modifying *scripts/all-measures/extract-all-measures.R*, one can then obtain all computed measures and save them in the same way as is done with the Toolkit's precomputed measures in said script. See the Full Toolkit Pipeline section for more.

Once relevant processing with the results is completed (for instance, indexing all of the measures from the `mvalues` datasets), the datasets can simply be removed with the following call:

```
rm(list=c(l1, l2, l3))
```

6.4 Parsing Probabilities Pipeline

Here, we walk through computing parsing probabilities for the full corpus and inspecting the results. For the relevant code, see *scripts/parsing-probabilities/parse-corpus-example.R*.

6.4.1 Running `parse_corpus`

With the Toolkit loaded and `all_words_PG` available, compute parsing probabilities for the entire corpus:

```
parsed_corpus <- parse_corpus(wordlist_v2_1_merged, all_words_PG)
```

This will take some time, as it builds a tree of all possible grapheme parsings for every word. A progress bar is displayed during computation.

6.4.2 Inspecting the Outputs

The four outputs can be inspected individually:

```
head(parsed_corpus[[1]]) # pp_table: letter-grapheme parsing probabilities
head(parsed_corpus[[2]]) # pp_reduced: reduced choice sets for pw_read
head(parsed_corpus[[3]]) # corpus_parsed: full word breakdown
head(parsed_corpus[[4]]) # corpus_pp: per-word PP summary
```

6.4.3 Querying PP for Specific Words

To see per-word PP summary statistics:

```
parsed_corpus[[4]][parsed_corpus[[4]]$spelling == "yacht", ]
parsed_corpus[[4]][parsed_corpus[[4]]$spelling == "reach", ]
```

As an example, YACHT will have a lower `pp_mean` than REACH because its grapheme parsings (e.g., A→[ACH]) are less common in the corpus compared to alternatives (A→[A], C→[CH]).

To see the per-letter PP breakdown:

```
parsed_corpus[[3]][parsed_corpus[[3]]$spelling == "yacht", ]
```

6.5 Generating Pseudoword Spellings and Pronunciations

The Toolkit provides two functions for working with pseudowords (or any novel spelling/pronunciation):

`pw_spell` generates possible spellings for a given pronunciation, and `pw_read` generates possible pronunciations for a given spelling. Both functions accept the `level` parameter to specify the grain size: "PG", "OC", "OR", or "all". Example scripts for both are available in *scripts/pw-fcns/*.

6.5.1 Generating Spellings with `pw_spell`

For the relevant code, see *scripts/pw-fcns/pw_spell_example.R*.

Get the most likely spelling for a pronunciation at the PG level:

```
pw_spell(target = "sib", level = "PG",
         PG_table = all_tables_PG, min_map = 1)
```

At the OC or OR level, pass the corresponding table:

```
pw_spell(target = "sib", level = "OC",
         OC_table = all_tables_OC, min_map = 1)
pw_spell(target = "sib", level = "OR",
         OR_table = all_tables_OR, min_map = 1)
```

To get the most likely spelling at all levels simultaneously:

```
pw_spell(target = "sib", level = "all",
         PG_table = all_tables_PG,
         OC_table = all_tables_OC,
```

```
OR_table = all_tables_OR, min_map = 1)
```

Setting `min_map` to a lower value (e.g., 0.01) returns all spellings above that probability threshold. Setting `score = FALSE` returns only the spelling options without scoring them.

6.5.2 Generating Pronunciations with `pw_read`

For the relevant code, see *scripts/pw-fcns/pw-read.example.R*.

The `pw_read` function generates possible pronunciations for a spelling based on the corpus statistics computed by `parse_corpus`.

Get the most likely pronunciation for a pseudoword at the PG level:

```
result <- pw_read(
  target      = "flope",
  parsed_corpus = parsed_corpus,
  PG_table    = all_tables_PG,
  min_map     = 1)
```

The result is a list with `phon_options` (possible phoneme sequences), `pp_means` (parsing probability means), and `gp_scores` (GP consistency scores).

Use `level="all"` to score at PG, OC, and OR levels simultaneously:

```
result <- pw_read(
  target      = "flope",
  parsed_corpus = parsed_corpus,
  PG_table    = all_tables_PG,
  OC_table    = all_tables_OC,
  OR_table    = all_tables_OR,
  level       = "all",
  score       = TRUE,
  min_map     = 1)
```

6.5.3 Adjusting Thresholds

Both `pw_spell` and `pw_read` use the `min_map` parameter to control how many results are returned:

- `min_map = 1`: Returns only the most likely result. Use this when you want a single best spelling or pronunciation.
- `min_map < 1` (e.g., 0.01): Returns all options whose probability exceeds the threshold. Lower values include more options.

`pw_read` has additional filtering parameters:

- `min_pp = 1` (default): Only the best (most probable) grapheme parse is used. Set to a value less than 1 to include alternative parses above the threshold.

- **mean_score**: Minimum mean score across all graphemes for the final pronunciation. Set to 0 (default) to return all, or 1 to return only the best.

Example with adjusted thresholds:

```
result <- pw_read(
  target      = "flope",
  parsed_corpus = parsed_corpus,
  PG_table    = all_tables_PG,
  OC_table    = all_tables_OC,
  OR_table    = all_tables_OR,
  level       = "all",
  score       = TRUE,
  min_map     = 0.5)
```

6.5.4 Interpreting pw_read Output

The output list contains:

- **result[[1]]**: Phoneme option sequences — each row is a possible pronunciation.
- **result[[2]]**: Mean PP values for each pronunciation.
- **result[[3]]** (if **score=TRUE**): GP scores at the requested level(s), including consistency and frequency measures from **map_value** and **summarize_words**.

6.6 Full Toolkit Pipeline

See *scripts/full-pipeline/full_pipeline.R*.

To adapt this script for any list of spellings, pronunciations, and frequency weights, only the relevant variable names at the top of the script need be changed.

The script first applies **replace_accented_vowels** to the spellings as a preprocessing step to strip any accented characters before mapping. The spelling/pronunciation/weight lists are then passed to the **batch** functions discussed in the previous section through the batch-mapping pipeline to produce datasets of the form indexed by the code in the measure extraction script. Code from this script is then adapted to index these measures and produce the *all_measures.csv* dataset. Finally, **parse_corpus** is called on the mapped words to compute parsing probabilities, and these PP measures are merged into the output.

All one needs to do is change the values of the relevant variables at the top of the script to index their specific dataset, as well as remove any of the available grain sizes and weight options if some are not required for processing. Undesired measures and statistics can also be omitted by removing them from the relevant lists.

6.7 Generating Pseudoword Measures

Here, we discuss how to generate all available measures from the Toolkit for a desired set of pseudowords (and their respective pronunciations). Obtaining these is slightly different from obtaining all measures from the Toolkit's preprocessed words, as measures involving those have been mapped and calculated prior; for pseudowords, each combination is novel and thus the Toolkit must compute the measures for the individual word on-the-fly based on its preprocessed dataset.

Code for this is available in

scripts/pseudowords/get_pseudoword_measures.R. We begin by defining some lists allowing us to iterate through all tables for each combination of weight and grain size, for each of the desired measures. As we iterate over these options for each pseudoword and its pronunciation, we need to *map* the word to its pronunciation at the current level:

```
mapped_pseudoword <- map_value(word, pron, level, table_data)
```

Where `table_data` corresponds to one of the tables (default, token-weighted, position-irrespective, token-weighted position-irrespective) at the current `level`. We then, for each available measure, pass the result into `summarize_words` to get all desired statistics:

```
summary <- summarize_words(mapped_pseudoword, measure)
```

You'll then see, in the next few lines, a few functions to manipulate the names of the obtained data for the purpose of including these in a final dataset similar to the one discussed in the previous section for all of the Toolkit measures.

6.8 Obtaining Number of Units

Here we discuss how to determine the number of units present in the Toolkit; i.e., the number of valid mappings at any given level. We also discuss how to obtain the individual numbers of unique phonemes and graphemes. This is useful as updates to the Toolkit often come with added mappings, which adjusts the total number at every level. Follow along in *scripts/all-units/get_all_units.R*.

The process for this is very straightforward; we first simply combine all datasets detailing valid mappings and take the number of unique phonemes and graphemes in the full set to obtain the total number of unique units for each.

For mappings at each individual level, we can simply take the number of mappings of the first word in the dataset, as measures are calculated for each mapping for each word. As such we iterate through the four and print the total number.

6.9 Finding Orthographic and Phonological Neighbors

It can be useful to find both the orthographic and phonological neighbors of both words and pseudowords. Here, we discuss the process for utilizing the Toolkit's datasets to find such neighbors at any of the four grain sizes (as well as the individual letter size in the orthographic case). Code for this is available in *scripts/neighbors/neighbor_finders.R*.

First, we define a function *edit_distance_matrices* for computing the edit distance (Levenshtein distance⁴). Note that this function takes in two matrices of the form given in the *all_tables* family of datasets, and is used whenever we are finding the neighbors at a level other than the individual letter level.

Next, we can define functions for computing the orthographic and phonological neighbors for words. At the orthographic level specifically, we may also be interested in simply the edit distance at the level of individual letters; the function defaults to finding such neighbors and is as simple as calling the *stringdist* function from the package of the same name. At all other levels, we call the function previously created between the words we are interested in and all of the words in the Toolkit's corpus; these are indexed based on the desired level (grain size). The syntax looks complex but is just indexing the vector in the table containing the orthographic representation of the word:

```
mygraphemes1 <-
  level_func(
    spelling=item_spellings[j],
    pronunciation=item_pronunciations[j])[[1]][[1]][2,]
```

The `[2,]` index marker at the end is particularly important as it indexes the orthographic representation itself; it turns out that finding the phonological neighbors is as simple as using the above edit distance function and indexing by `[1,]` instead. Also note that here, *level_func* corresponds to the relevant function from the Toolkit for mapping the word(s) we are interested in at the desired level; for instance, if we would like to find the ONC-neighbors of a word, we must first map that word at the ONC level.

Once we've obtained the graphemes for the word itself, we use the edit distance function to compute the distances between the word and all corpus words. We then simply return all words satisfying the condition that the edit distance is less than the threshold (which defaults to 1).

At this point, it is useful to discuss what this distance actually refers to when the level is not the individual letter. When we discuss insertion, deletion, and substitution of graphemic/phonemic units in such a case, we refer to the insertion, deletion, and substitution of an entire unit, which may be more than one letter. This is why the mapping of the word itself is important as letters and phonemes can be grouped together in different ways (and thus yield different neighbors at different levels).

As mentioned before, the code for obtaining phonological neighbors is nearly identical, except here the tables are indexed differently in order to obtain the phonological, rather than orthographic, representation. The same edit distance function suffices.

⁴The standard metric for determining the distance between two strings as a function of insertions, deletions, and substitutions; see https://en.wikipedia.org/wiki/Levenshtein_distance

7 Troubleshooting

This section will cover some common problems that occur when using the Toolkit, and how to identify their causes.

7.1 Function Hangs without Error

Sometimes, when processing words with certain Toolkit functions in batch (i.e. passing in lists of items at once rather than individual strings), the Toolkit will "hang" instead of completely erroring and concluding program execution. When this happens, iterating through each possible problem individually with a `tryCatch` block allows one to examine each error.

Example 7.1. Consider a dataset *my_pseudowords* containing two columns for spelling and pronunciation: *spell* and *pron*. We'd like to map all of these spelling-pronunciation pairs at the ONC level and summarize them using the `summarize_words` function. However, if there are one or more mappings that would be invalid in this dataset (e.g. not accepted by the Toolkit), then running the following line which would otherwise work:

```
summarize_words(
  map_value(
    my_pseudowords$spelling,
    my_pseudowords$pron,
    "ONC",
    all_tables_ONC),
  "PG")
```

will instead hang indefinitely in the R terminal. To find the problematic entries, we make use of a simple `tryCatch` block in a loop that maps and summarizes words in the dataset one-by-one (where *s* and *p* are strings), collecting the errors in a list (erroneous entries causing the program to hang are guaranteed to throw a visible error when examined individually):

```
summary <- tryCatch(
  {
    summarize_words(map_value(s, p, "ONC", all_tables_ONC), "PG")
  },
  error=function(e) {
    error_df <- rbind(error_df, data.frame(spell = s, pron = p))
  })
```

In the above, `error_df` has a *spell* and *pron* column where erroneous entries are collected. If there is no error for an entry, we instead have the result of the call in the `summary` variable.

The full template code for this is available on the GitHub. See *scripts/troubleshooting/hanging/identifying-problematic-entries.R*.

8 Version 2.1 Updates

The following changes were introduced in Toolkit version 2.1. These updates simplify several function interfaces and add new capabilities.

8.1 New Wordlist

The corpus is now `wordlist_v2_1_merged` (22,257 words), which replaces `wordlist_v2_0`. All scripts and function calls should reference the new name.

8.2 `summarize_words` — ONC Mode

`summarize_words` gains a `mode` parameter. Setting `mode = "ONC"` provides a breakdown of consistency by syllabic position (onset, nucleus, coda) rather than summarizing across all units:

```
summarize_words(
  mapped_words = scored_words_PG[
    which(wordlist_v2_1_merged$spelling == "penguin")],
  parameter = "PG",
  mode = "ONC")
```

8.3 Simplified `min_map` Parameter

`pw_spell()` now uses `min_map` in place of the previous `minimum` parameter. `pw_read()` now uses `min_map` and `mean_score` in place of the previous `min_gp` and `mean_gp` parameters:

- `min_map = 1`: Returns only the most likely result (equivalent to old `minimum=1` or `min_gp=1`).
- `min_map = 0.01`: Returns all options with at least 1% probability.
- `min_map = 0`: Returns all possible options.

8.4 `visualize_parse_tree` (New Function)

`visualize_parse_tree(spelling, pronunciation, show_plot)` visualizes how a spelling can be parsed into graphemes, optionally highlighting the parse path that corresponds to a given pronunciation. For the relevant code, see *scripts/visualization/visualize_parse_tree_example.R*.

```
# Show all possible grapheme parsings for "sebe"
visualize_parse_tree("sebe")

# Highlight the path matching pronunciation "sib"
visualize_parse_tree("sebe", "sib", show_plot = TRUE)

# Highlight the path matching pronunciation "sibi"
visualize_parse_tree("sebe", "sibi", show_plot = TRUE)
```

9 Appendix

9.1 Phoneme Reference

The following is a table illustrating the conversions between the IPA pronunciation format and the Toolkit's conventions.

IPA	Toolkit	Example
ɑ, ɒ	a	father <i>faDʒr</i>
æ	@	cat <i>k@t</i>
ʌ	^	mutt <i>m ^t</i>
ə	e	about <i>ebOt</i>
ɔ	c	force <i>fcrs</i>
aʊ	O	wow <i>wO</i>
aɪ	5	light <i>l5t</i>
ɛ	E	net <i>nEt</i>
ɜ, ɝ	3r	third <i>T3rd</i>
eɪ	8	weight <i>w8t</i>
ɪ	1	fit <i>f1t</i>
i	i	neato <i>nito</i>
oʊ	o	ghost <i>gost</i>
ɔɪ	2	poise <i>p2z</i>
ʊ	U	look <i>luK</i>
u	u	compute <i>kempjut</i>
b	b	baby <i>b8bi</i>
tʃ	C	watch <i>waC</i>
d	d	dude <i>dud</i>
ð	D	brother <i>br ^Dʒr</i>

IPA	Toolkit	Example
f	f	fig <i>f1g</i>
g	g	igloo <i>1glu</i>
h	h	hello <i>hElo</i>
dʒ	G	juggle <i>G ^gel</i>
k	k	keep <i>kip</i>
l	l	lazy <i>l8zi</i>
m	m	money <i>m ^ni</i>
n	n	night <i>n5t</i>
ŋ	N	wing <i>w1N</i>
p	p	please <i>pliz</i>
ɪ	r	reach <i>riC</i>
s	s	scythe <i>s5D</i>
ʃ	S	shell <i>SEl</i>
t	t	twin <i>tw1n</i>
θ	T	think <i>T1Nk</i>
v	v	very <i>vEri</i>
w	w	willow <i>w1lo</i>
j	j	you <i>ju</i>
z	z	maze <i>m8z</i>
ʒ	Z	pleasure <i>plEZʒr</i>